Storybook + MSW + Visual Regression — Starter Guide

Target repo: `apps/web` (Next.js 14, TypeScript, Tailwind 3.4, pnpm 9)
Goal: setup Storybook for UI components, integrate MSW (Mock Service Worker) so stories mock API calls, and add automated visual regression tests using Playwright.
Versions (locked): Storybook `^7.7.x` (use latest 7.x), MSW `^1.2.x`, Playwright `^1.46.x`. Adjust exact patch via lockfile.

Dokumen ini berisi: requirements, install steps, config files, example component + story, MSW handlers, Playwright visual tests, CI pipeline example, and troubleshooting.

---

0. Filosofi

- Komponen dibangun *component-first* (Storybook + Story-driven development).
- Semua network interactions di stories dimock lewat MSW sehingga designer/QA/dev bisa iterate tanpa backend.
- Visual regression dijalankan terhadap Storybook UI — deterministic, environment-controlled snapshots.

---

1. Prasyarat

- Node 20.x active (`.nvmrc` present)
- pnpm 9
- Existing Next app structure under `apps/web` with `src/` and `app/` routes
- Tailwind & shadcn/ui already present (from previous setup)

---

2. Installasi paket

Jalankan di root monorepo atau di `apps/web` (direkomendasikan di package scope `apps/web`):

```
cd apps/web
pnpm add -D @storybook/cli@7 @storybook/react-webpack@7 @storybook/addon-
essentials@7 @storybook/test-runner @storybook/addon-interactions
pnpm add -D msw@1.2 @storybook/addon-msw
pnpm add -D @playwright/test@1.46 playwright-ct @playwright/test-runner
pnpm add -D jest-image-snapshot#optional
```

Catatan: gunakan `@storybook/react-webpack` builder karena kompatibel dengan Next internals; jika pilih Vite, ada langkah setup tambahan. Webpack builder biasanya lebih straightforward with Next's server components; pick webpack unless team wants Vite.

---

3. Storybook init & config

1. Jalankan awal (opsional interactive):

```
pnpm dlx sb@7 init --type react_scripts
```

2. Manual minimal config files (diletakkan di `apps/web/.storybook/`):

main.ts

```ts
import type { StorybookConfig } from '@storybook/react-webpack'
const config: StorybookConfig = {
  stories: ['../src/components/**/*.stories.@(ts|tsx)'],
  staticDirs: ['../public'],
  addons: [
    '@storybook/addon-essentials',
    '@storybook/addon-interactions',
    '@storybook/addon-msw'
  ],
  framework: {
    name: '@storybook/react-webpack',
    options: {}
  },
  features: { storyStoreV7: true }
}
export default config
```

preview.tsx (global decorators & MSW setup)

```tsx
import React from 'react'
import { Preview } from '@storybook/react'
import { initialize, mswDecorator } from 'msw-storybook-addon'
import '../src/styles/globals.css' // tailwind

// initialize MSW
initialize()

const preview: Preview = {
  decorators: [mswDecorator],
```

```
  parameters: {
    actions: { argTypesRegex: '^on[A-Z].*' },
    controls: { expanded: true }
  }
}

export default preview
```

manager.ts (optional theming)

```
import { addons } from '@storybook/addons'
import { themes } from '@storybook/theming'
addons.setConfig({ theme: themes.light })
```

---

4. Tailwind integration

Pastikan Storybook memuat Tailwind. Di `apps/web/.storybook/preview.tsx` sudah import `globals.css`. Pastikan `globals.css` memuat `@tailwind base; @tailwind components; @tailwind utilities;`.

Jika PostCSS/Tailwind tidak bekerja, tambahkan `webpackFinal` override in `main.ts`:

```
// inside main.ts
webpackFinal: async (config) => {
  // ensure css loaders handled similar to project
  return config
}
```

---

5. Contoh Komponen + Story

Bikin folder `apps/web/src/components/ui/Button.tsx`:

```
import React from 'react'

export type ButtonProps = React.ButtonHTMLAttributes<HTMLButtonElement> & {
  variant?: 'primary' | 'secondary'
}

export function Button({ variant = 'primary', children, ...rest }: ButtonProps)
```

```
{
  const base = 'px-4 py-2 rounded-md font-medium'
  const cls = variant === 'primary' ? `${base} bg-blue-600 text-white` : `$
{base} bg-gray-200 text-gray-900`
  return (
    <button className={cls} {...rest}>
      {children}
    </button>
  )
}
export default Button
```

Buat story `apps/web/src/components/ui/Button.stories.tsx`:

```
import React from 'react'
import type { Meta, StoryObj } from '@storybook/react'
import { Button } from './Button'

const meta: Meta<typeof Button> = { title: 'UI/Button', component: Button }
export default meta

type Story = StoryObj<typeof Button>
export const Primary: Story = { args: { children: 'Primary Button', variant:
'primary' } }
export const Secondary: Story = { args: { children: 'Secondary', variant:
'secondary' } }
```

---

6. MSW handlers (mock API)

Buat folder `apps/web/.storybook/mocks/` dan file `handlers.ts`:

```
import { rest } from 'msw'

export const handlers = [
  rest.get('http://localhost:3001/api/v1/employees', (req, res, ctx) => {
    return res(ctx.status(200), ctx.json({ total: 1, items: [{ id: 'e-1',
employeeNo: 'E-0001', firstName: 'Budi' }] }))
  })
]
```

Lalu update `preview.tsx` untuk register handlers default:

```
import { handlers } from './mocks/handlers'
initialize({ onUnhandledRequest: 'bypass' })
// pass default handlers so stories use them
export const parameters = { msw: { handlers } }
```

Catatan: stories can override handlers per story via `decorators` or `parameters.msw`.

---

7. Storybook run scripts

Di `apps/web/package.json` tambahkan:

```
"scripts": {
  "storybook": "storybook dev -p 6006",
  "build-storybook": "storybook build",
  "test-storybook": "storybook test"
}
```

Jalankan lokal: `pnpm storybook` → buka `http://localhost:6006`.

---

8. Visual Regression with Playwright

Kita gunakan Playwright to render Storybook stories and snapshot them. Playwright CT is for component testing, but here we will run Playwright against the built Storybook static site (deterministic backgrounds). Steps:

8.1 Install Playwright (already in deps)

```
pnpm dlx playwright@1.46 install --with-deps
```

8.2 Playwright config (`apps/web/playwright.config.ts`)

```
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: 'playwright',
  timeout: 30_000,
  expect: { toHaveScreenshot: { maxDiffPixelRatio: 0.0001 } },
  fullyParallel: true,
  reporter: [['list'], ['html', { open: 'never' }]],
  use: { baseURL: 'http://localhost:6006', trace: 'on-first-retry', headless:
```

```
true },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } }
  ]
})
```

8.3 Playwright test file (snapshot) `apps/web/playwright/tests/visual.spec.ts`

```
import { test, expect } from '@playwright/test'

const stories = [
  { path: '/iframe.html?id=ui-button--primary', name: 'button-primary' },
  { path: '/iframe.html?id=ui-button--secondary', name: 'button-secondary' }
]

for (const s of stories) {
  test(`visual: ${s.name}`, async ({ page }) => {
    await page.goto(s.path)
    const root = page.locator('#root') // Storybook mounts story in #root
    await expect(root).toHaveScreenshot(`${s.name}.png`, { animations:
'disabled' })
  })
}
```

Catatan: Storybook iframe path format `iframe.html?id=story-id` where `story-id` is kebab of story title and story name.

8.4 Local run (dev)

1. Start Storybook: `pnpm storybook` (port 6006).
2. In another terminal: `pnpm playwright test`.
   Playwright will create snapshot images under `playwright/test-results` (or configured folder).
   The first run creates baseline images. Subsequent runs compare and fail on diffs.

---

9. CI: GitHub Actions example (visual regression on PR)

Create `.github/workflows/visual-regression.yml` (root) with step to build storybook and run playwright tests.

```
name: Visual Regression
on: [pull_request]
jobs:
```

```
  visual:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Use Node
        uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Install pnpm
        run: npm i -g pnpm@9
      - name: Install deps
        run: pnpm install --frozen-lockfile
        working-directory: apps/web
      - name: Build Storybook
        run: pnpm build-storybook --output-dir storybook-static
        working-directory: apps/web
      - name: Start Storybook
        run: npx http-server ./storybook-static -p 6006 &
        working-directory: apps/web
      - name: Install Playwright browsers
        run: pnpm dlx playwright@1.46 install --with-deps
        working-directory: apps/web
      - name: Run Playwright Visual Tests
        run: pnpm playwright test --reporter=list
        working-directory: apps/web
      - name: Upload diff artifacts
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: vr-diffs
          path: apps/web/playwright/test-results
```

Notes: - First PR baseline will create images — you may want to accept them via baseline approval process. Alternatively, run baseline job once on `main` to populate baseline. - You can store baselines in a separate branch or artifact store and pull them in CI job for PR comparison.

---

10. Baseline management & recommendations

- Keep baseline images in a dedicated folder under `apps/web/playwright/baseline/` and commit them (only if small).
- Preferred pattern: baselines published as release artifact from `main` build and downloaded into PR job for comparisons. This avoids merge conflicts on large binary files.
- If visual diffs are expected (intentional UI change), use a process to accept updated baseline (QA reviewer approves update job that replaces baseline artifacts).

---

11. Storybook Testing Library integration (optional)

Install `@storybook/testing-library` and write interaction tests per story that can be executed as unit tests.

---

12. Acceptance Criteria (what 'done' looks like)

- `pnpm storybook` runs locally and shows Button stories.
- MSW successfully mocks `GET /api/v1/employees` and stories consuming API render stable data.
- `pnpm playwright test` runs against Storybook and creates baseline screenshots on first run.
- GitHub Actions runs build + test on PR and fails when visual diffs exceed threshold.

---

13. Troubleshooting & tips

- **Flickering / animations**: disable animations in stories (add global CSS to set `* { animation: none !important; transition: none !important }` in test preview) or use `await page.waitForTimeout(100)` before screenshot.
- **Fonts**: load consistent fonts in Storybook (use system fonts or bundle webfont in staticDir) so snapshots are deterministic.
- **Env differences**: ensure Storybook static build uses deterministic env variables (set `NODE_ENV=production` and same CSS variables).
- **Non-deterministic content**: seed random IDs/time with deterministic values in stories or mock clocks.

---

14. Next improvements (roadmap)

- Integrate Chromatic (paid) for cloud visual regression & review UI.
- Add Percy or Applitools for advanced visual diffs (AI smart diffs).
- Add per-component accessibility tests via `axe` in storybook test runner.
- Use Storybook Design Tokens and automated theme snapshot matrix (light/dark/large-font).

If mau, gue bisa langsung generate and commit the actual files into the repo (storybook config, example component, handlers, Playwright tests) — pilih *generate files* dan gue buat semua file ready-to-commit dalam canvas atau sebagai patch yang bisa kamu paste ke repo.